

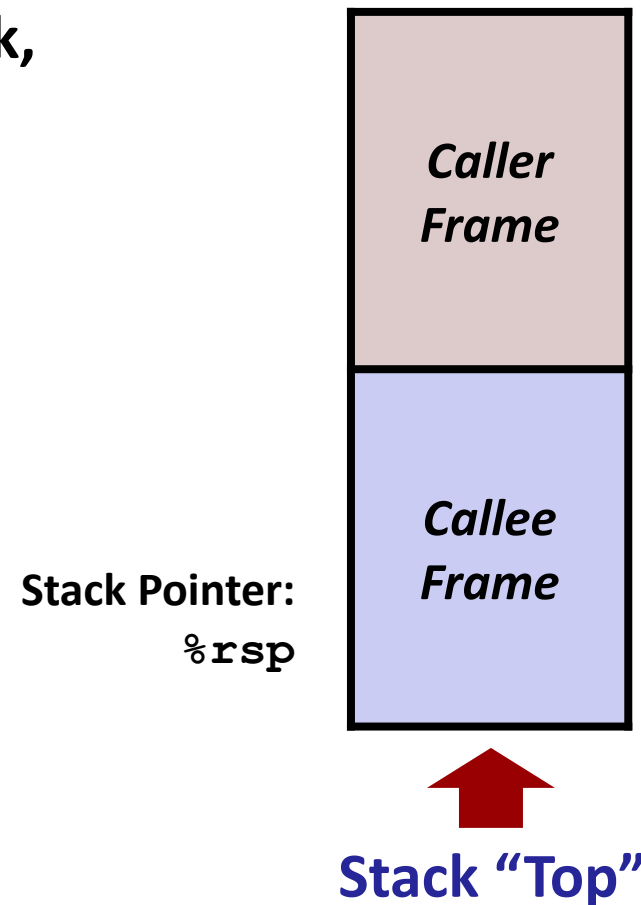
Machine-Level Programming V: Functions and the call stack

cs144

Prof. Yanjing Li

The Stack

- Each “function” has its own space on stack, called a **stack frame**
- **Stack frame management**
 - Allocated when enter procedure (“**call**” code)
 - Deallocated when returns (“**ret**” code)
 - `%rsp` must be put back to the end of Caller Frame when Callee returns
- **Last-in first-out” (LIFO) stack discipline matches function call/ret patterns**
 - If P calls Q, then Q returns before P





Today

■ Procedures

■ ~~Stack Structure~~

■ Calling Conventions

- Passing control
- Passing data
- Managing local data

Procedure Control Flow

- **Procedure call: `call label`**
 - Push **return address** on stack
 - Address of the next instruction right after call
 - Jump to *label*
- **Procedure return: `ret`**
 - Pop address from stack
 - Jump to address



Call/Ret Example

```
void multstore
(long x, long y, long
*dest){
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
400540: push    %rbx        # Save %rbx
400541: mov     %rdx,%rbx    # Save dest
400544: callq  400550 <mult2> # mult2(x,y)
400549: mov     %rax,(%rdx)  # Save at dest
40054c: pop     %rbx        # Restore %rbx
40054d: ret                     # Return
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
400550: mov     %rdi,%rax    # a
400553: imul    %rsi,%rax    # a * b
400557: ret                     # Return
```

Call/Ret Example

```
void multstore
(long x, long y, long
*dest){
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
400540: push    %rbx        # Save %rbx
400541: mov     %rdx,%rbx    # Save dest
400544: callq  400550 <mult2> # mult2(x,y)
400549: mov     %rax, (%rdx)  # Save at dest
40054c: pop     %rbx        # Restore %rbx
40054d: ret                # Return
```

*we need to
come back to
here*

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
400550: mov     %rdi,%rax    # a
400553: imul    %rsi,%rax    # a * b
400557: ret                # Return
```

where we go

*control changes to first element
of the control*



Call/Ret Example

```
void multstore
(long x, long y, long
*dest){
    long t = mult2(x, y);
    *dest = t;
}
```

```
00000000000400540 <multstore>:
400540: push    %rbx        # Save %rbx
400541: mov     %rdx,%rbx    # Save dest
400544: callq  400550 <mult2> # mult2(x,y)
400549: mov     %rax,(%rdx)  # Save at dest
40054c: pop     %rbx        # Restore %rbx
40054d: ret                # Return
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
00000000000400550 <mult2>:
400550: mov     %rdi,%rax    # a
400553: imul    %rsi,%rax    # a * b
400557: ret                # Return
```



Call/Ret Example

```
00000000000400540 <multstore>:
```

```
•  
•  
400544: callq 400550 <mult2>  
400549: mov %rax, (%rdx)  
•  
•
```

```
00000000000400550 <mult2>:
```

```
400550: mov %rdi,%rax  
•  
•  
400557: ret
```

0x130

0x128

0x120

%rsp

0x120

Stack
Pointer

%rip

0x400544

Next instruction
of caller

*which instruction
were about to
execute*

%rip: Program Counter
(PC)



Call/Ret Example

```
00000000000400540 <multstore>:
```

```
•  
•  
400544: callq 400550 <mult2>  
400549: mov %rax, (%rdx)  
•  
•
```

```
00000000000400550 <mult2>:
```

```
400550: mov %rdi,%rax  
•  
•  
400557: ret
```

0x130

0x128

0x120

0x118

0x400549

%rsp

0x118

%rip

0x400550

Return address
saved here!!

Stack
Pointer

Next instruction
now in callee

%rip: Program Counter
(PC)



Call/Ret Example

```
00000000000400540 <multstore>:
```

•
•
•

```
400544: callq 400550 <mult2>
```

```
400549: mov %rax, (%rdx) ←
```

•
•

```
00000000000400550 <mult2>:
```

```
400550: mov %rdi, %rax
```

•
•

```
400557: ret ←
```

0x130

0x128

0x120

0x118

0x400549

%rsp

0x118

%rip

0x400557

Return address

Stack
Pointer

Next instruction
now in callee

%rip: Program Counter
(PC)



Call/Ret Example

```
00000000000400540 <multstore>:  
.  
.  
400544: callq 400550 <mult2>  
400549: mov %rax, (%rdx)  
.  
.
```

```
00000000000400550 <mult2>:  
400550: mov %rdi,%rax  
.  
.  
400557: ret
```

0x130

0x128

0x120

*then it
goes back
to change
the
stack*

%rsp

0x120

Stack
Pointer

%rip

0x400549

**Next instruction
= return addr!**

%rip: Program Counter
(PC)



Today

■ Procedures

■ ~~Stack Structure~~

■ Calling Conventions

■ ~~Passing control~~

■ Passing data

■ Managing local data

Procedure Data Flow

Registers

■ First 6 arguments

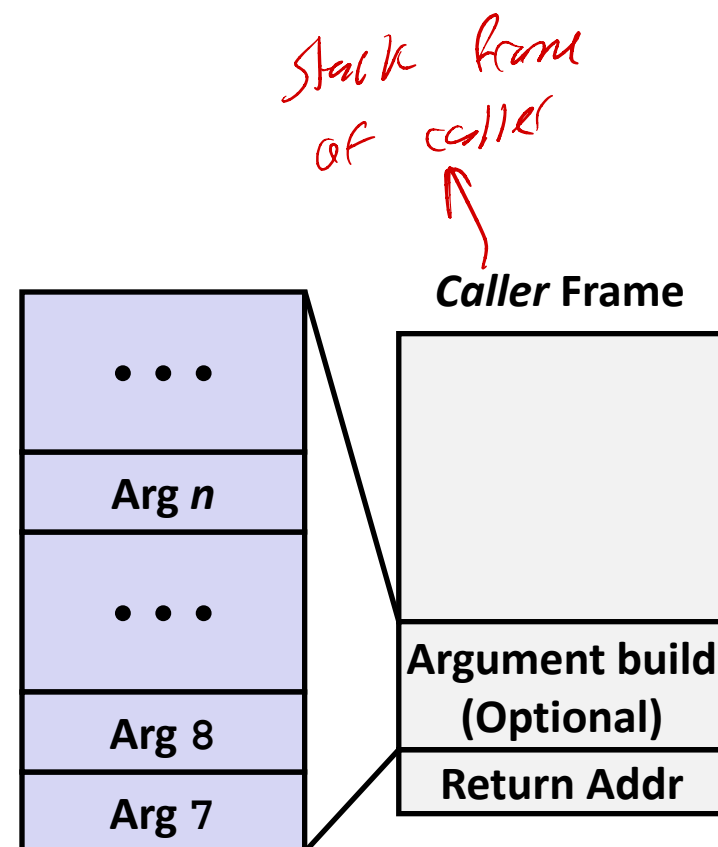
%rdi
%rsi
%rdx
%rcx
%r8
%r9

■ Return value

%rax

Stack

*caller
frame has
argument
builder*



- Only allocate stack space when registers are not enough



Data flow examples

```
void multstore
(long x, long y, long
*dest){
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
# x in %rdi, y in %rsi, dest in %rdx
400540: push    %rbx        # Save %rbx
400541: mov     %rdx,%rbx    # Save dest
400544: callq   400550 <mult2>    # mult2(x,y)
# t in %rax
400549: mov     %rax,(%rdx)  # Save at dest
40054c: pop     %rbx        # Restore %rbx
40054d: ret                # Return
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
# a in %rdi, b in %rsi
400550: mov     %rdi,%rax    # a
400553: imul    %rsi,%rax    # a * b
# s in %rax
400557: ret                # Return
```



Data flow examples

```
void multstore
(long x, long y, long
*dest){
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
# x in %rdi, y in %rsi, dest in %rdx
400540: push    %rbx        # Save %rbx
400541: mov     %rdx,%rbx    # Save dest
400544: callq   400550 <mult2> # mult2(x,y)
# t in %rax
400549: mov     %rax, (%rdx) # Save at dest
40054c: pop     %rbx        # Restore %rbx
40054d: ret                # Return
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
# a in %rdi, b in %rsi
400550: mov     %rdi,%rax    # a
400553: imul    %rsi,%rax    # a * b
# s in %rax
400557: ret                # Return
```



Today

■ Procedures

■ ~~Stack Structure~~

■ Calling Conventions

■ ~~Passing control~~

■ ~~Passing data~~

■ Managing local data

Register Saving Conventions

- Each function has its private stack frame. But registers are shared.

- When procedure `yoo` calls `who`:

- `yoo` is the *caller*
- `who` is the *callee*

*We can have many
many function, but only
one register*

```
yoo:
    . . .
    movq $60637, %rdx
    call who
    addq %rdx, %rax
    . . .
    ret
```

```
who:
    . . .
    subq $15213, %rdx
    . . .
    ret
```

- Contents of register `%rdx` overwritten by `who`
- This could be trouble → something should be done!
 - Need some coordination

Register Saving Conventions

■ When procedure yoo calls who:

- yoo is the **caller**
- who is the **callee**

Save contents on stack

■ Conventions

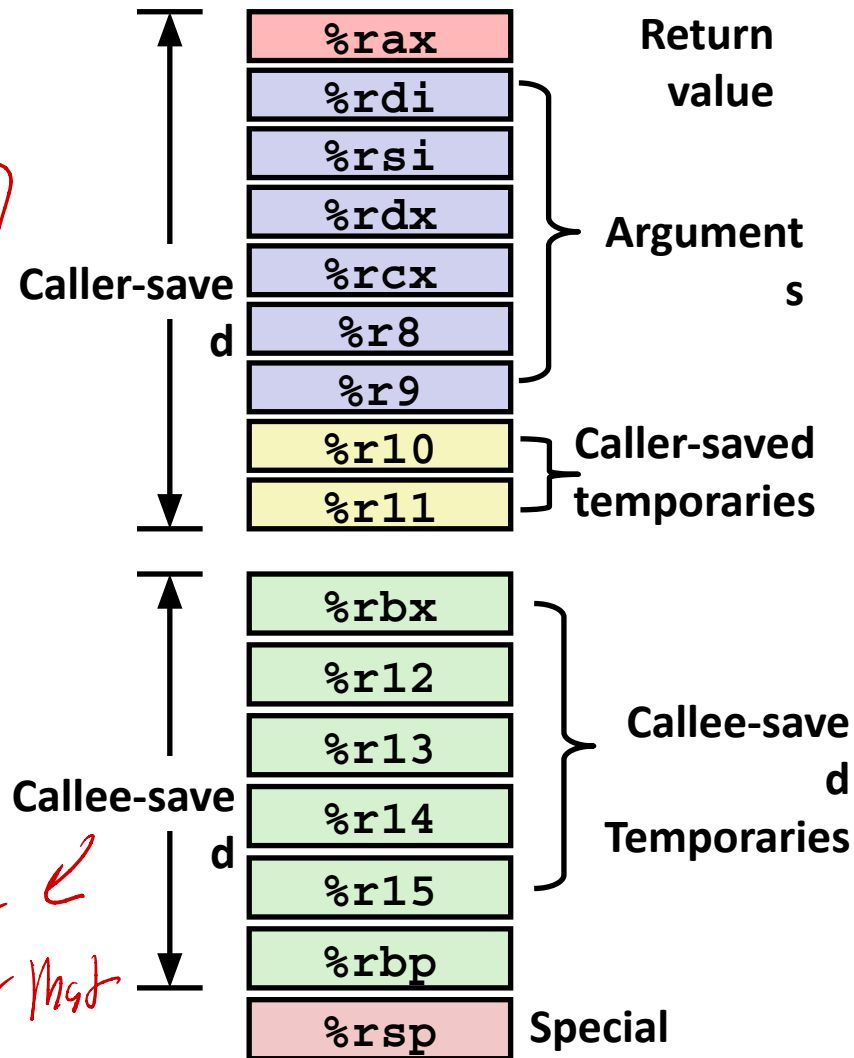
■ “Caller Saved”

- Caller saves temporary values in its frame before the call

■ “Callee Saved”

- Callee saves temporary values in its frame before using
- Callee restores them before returning to caller

returns the values that the caller saved on a stack



Procedure Assembly Example I

```
void multstore
(long x, long y, long
*dest){
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
400540: push    %rbx      # Save %rbx
400541: mov     %rdx,%rbx  # Save dest
400544: callq  400550 <mult2>    # mult2(x,y)
400549: mov     %rax,(%rdx) # Save at dest
40054c: pop     %rbx      # Restore %rbx
40054d: ret          # Return
```

*callq save register by pushing on stack
we know we want the val in rdx*

dest = t

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
400550: mov     %rdi,%rax  # a
400553: imul    %rsi,%rax  # a * b
400557: ret          # Return
```

Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
incr:  
    movq     (%rdi), %rax  
    addq     %rax, %rsi  
    movq     %rsi, (%rdi)  
    ret
```

Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

```
incr:  
    movq     (%rdi), %rax  
    addq     %rax, %rsi  
    movq     %rsi, (%rdi)  
    ret
```

*goes right
back*

Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
incr:  
    movq     (%rdi), %rax  
    addq     %rax, %rsi  
    movq     %rsi, (%rdi)  
    ret
```

Register	Use(s)
%rdi	Argument p
%rsi	Argument val
%rax	x , Return value

we need to return
with rax

Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

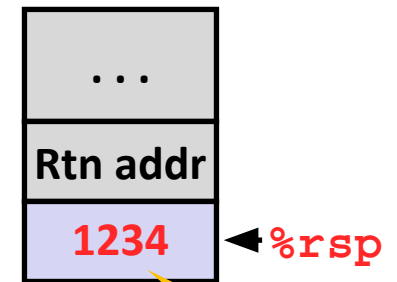
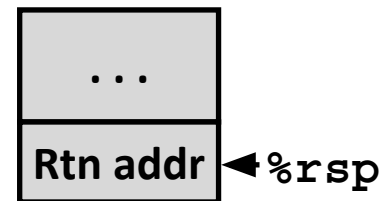
```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

Initial
states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	

Resulting states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	



*we push rbx
because rbx is callee save,
so we have to make sure that
we save the rbx*

Saved
%rbx

Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

Procedure Assembly Example II

```
long call_incr2(long x) {
    long v1 = 60637;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

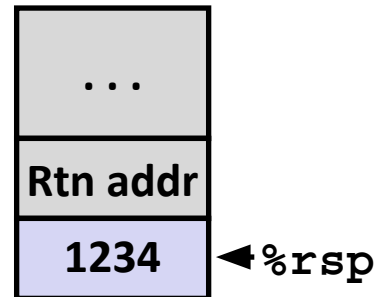
```
call_incr2:
    pushq    %rbx
    subq     $16, %rsp
    movq     $60637, 8(%rsp)
    movq     %rdi, %rbx
    movq     $3000, %rsi
    leaq     8(%rsp), %rdi
    call     incr
    addq     %rbx, %rax
    addq     $16, %rsp
    popq     %rbx
    ret
```

Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

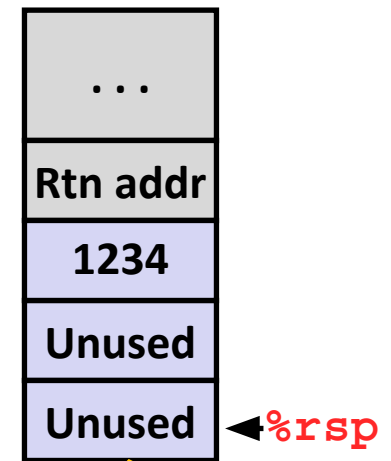
Initial
states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	



Resulting states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	



Each box is a word
(8bytes)

Procedure Assembly Example II

```
long call_incr2(long x) {
    long v1 = 60637;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

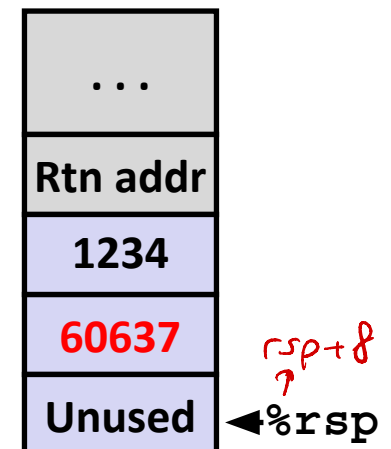
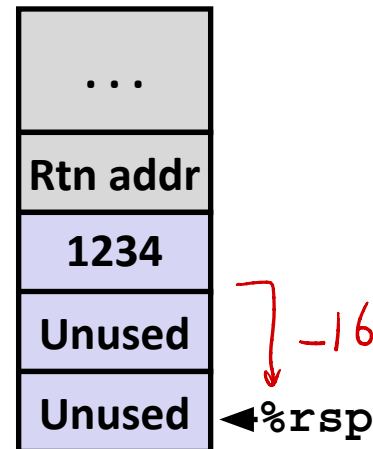
```
call_incr2:
    pushq    %rbx
    subq     $16, %rsp
    movq     $60637, 8(%rsp)
    movq     %rdi, %rbx
    movq     $3000, %rsi
    leaq     8(%rsp), %rdi
    call     incr
    addq     %rbx, %rax
    addq     $16, %rsp
    popq     %rbx
    ret
```

Initial
states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	

Resulting states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	



Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

we need to wipe this out

We know v1 needs to be in memory

Procedure Assembly Example II

```
long call_incr2(long x) {
    long v1 = 60637;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

```
call_incr2:
    pushq    %rbx
    subq     $16, %rsp
    movq     $60637, 8(%rsp)
    movq     %rdi, %rbx
    movq     $3000, %rsi
    leaq     8(%rsp), %rdi
    call     incr
    addq     %rbx, %rax
    addq     $16, %rsp
    popq     %rbx
    ret
```

Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

Initial
states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	1234
%rax	

Resulting states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	x
%rax	

Original %rbx was
saved on stack

...
Rtn addr
1234
60637
Unused

*We know
that rbx is
callee save so
we can get
it back*

after our call

Rtn addr
1234
60637
Unused

←%rsp

Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

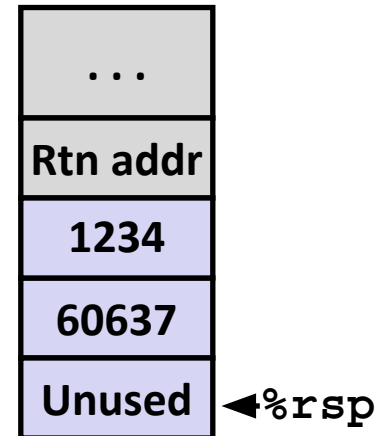
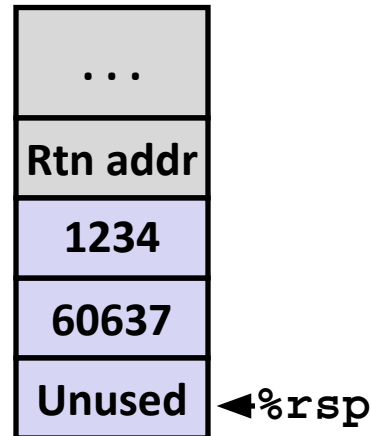
```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

Initial
states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	
%rbx	x
%rax	

Resulting states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	3000
%rbx	x
%rax	



Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

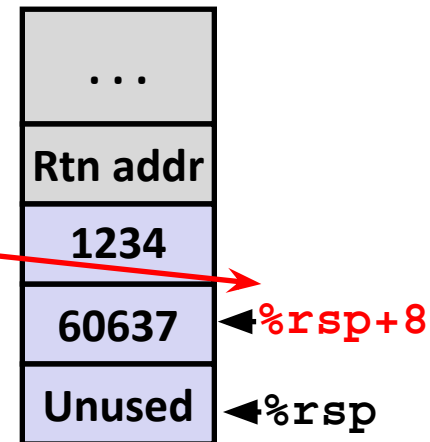
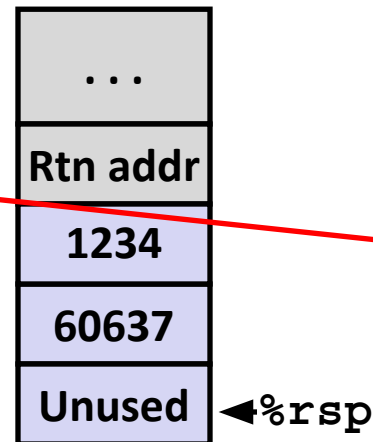
```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

Initial
states

Register	Use(s)
%rdi	x (1 st arg)
%rsi	3000
%rbx	x
%rax	

Resulting states

Register	Use(s)
%rdi	&v1
%rsi	3000
%rbx	x
%rax	



Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

Procedure Assembly Example II

```
long call_incr2(long x) {
    long v1 = 60637;
    long v2 = incr(&v1, 3000);
    return x+v2;
}
```

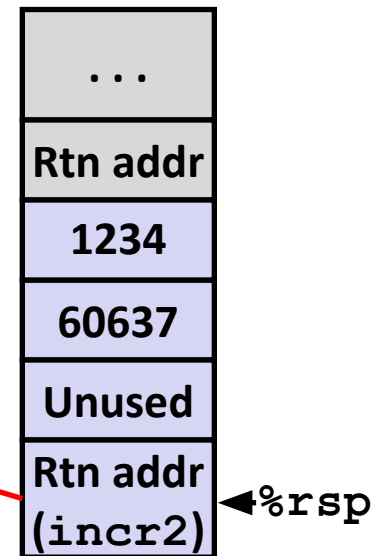
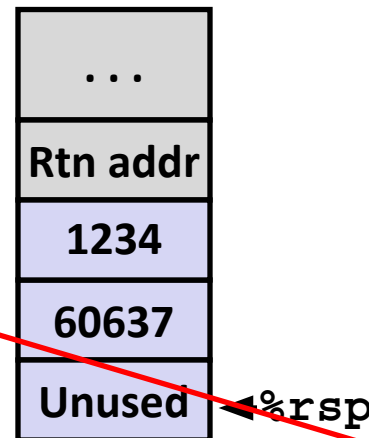
```
call_incr2:
    pushq    %rbx
    subq     $16, %rsp
    movq     $60637, 8(%rsp)
    movq     %rdi, %rbx
    movq     $3000, %rsi
    leaq     8(%rsp), %rdi
    call     incr
    addq     %rbx, %rax
    addq     $16, %rsp
    popq     %rbx
    ret
```

Initial
states

Register	Use(s)
%rdi	&v1
%rsi	3000
%rbx	x
%rax	

Resulting states

Register	Use(s)
%rdi	&v1
%rsi	3000
%rbx	x
%rax	



Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

*push return
address on stack*

Procedure Assembly Example II

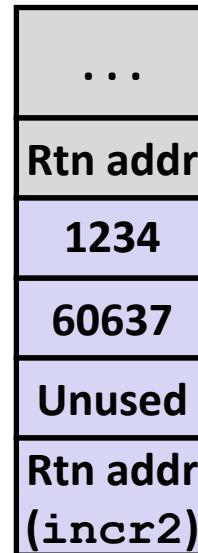
```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
incr:  
    movq    (%rdi), %rax  
    addq    %rax, %rsi  
    movq    %rsi, (%rdi)  
    ret
```

Caller-saved: %rdi, %rsi, %rax
Callee-saved: %rbx

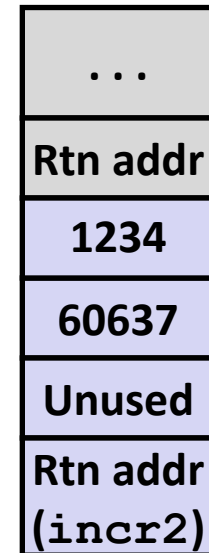
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	3000
%rbx	x
%rax	



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	3000
%rbx	x
%rax	v1 (60637)



^
return
value in
rax

Procedure Assembly Example II

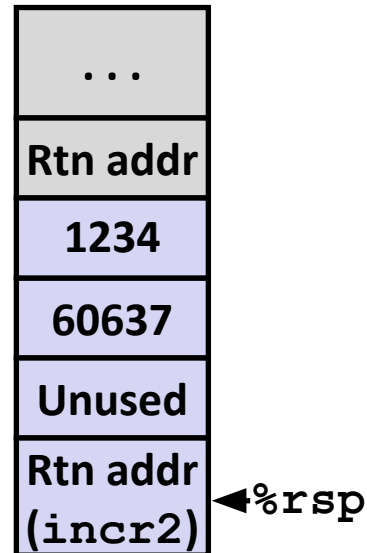
```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
incr:  
    movq    (%rdi), %rax  
    addq    %rax, %rsi  
    movq    %rsi, (%rdi)  
    ret
```

Caller-saved: %rdi, %rsi, %rax
Callee-saved: %rbx

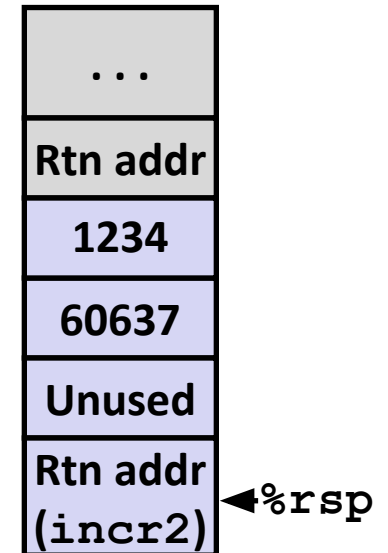
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	3000
%rbx	x
%rax	v1 (60637)



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	y (=x+val)
%rbx	x
%rax	v1 (60637)



Procedure Assembly Example II

```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

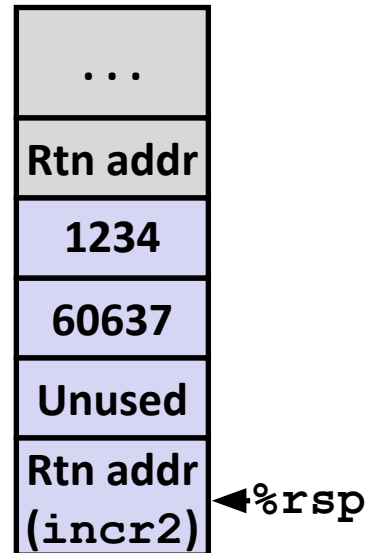
```
incr:  
    movq    (%rdi), %rax  
    addq    %rax, %rsi  
    movq    %rsi, (%rdi)  
    ret
```

*how to go back to
its caller*

Caller-saved: %rdi, %rsi, %rax
Callee-saved: %rbx

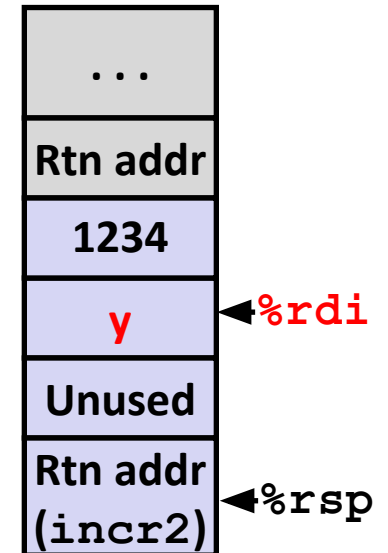
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	v1 (60637)



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	v1 (60637)



Procedure Assembly Example II

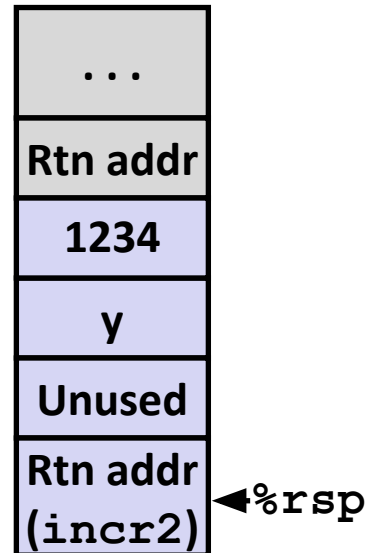
```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
incr:  
    movq    (%rdi), %rax  
    addq    %rax, %rsi  
    movq    %rsi, (%rdi)  
    ret
```

Caller-saved: %rdi, %rsi, %rax
Callee-saved: %rbx

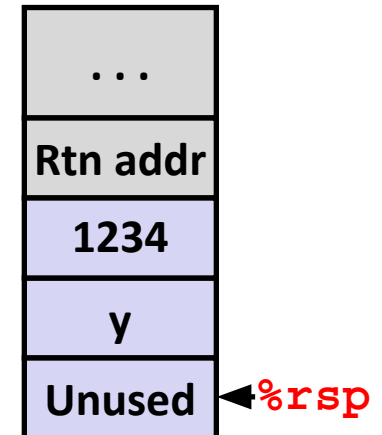
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	v1 (60637)



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	v1 (60637)



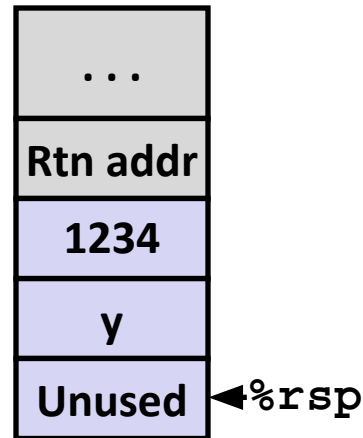
Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

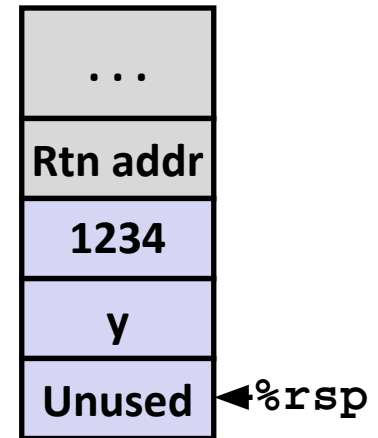
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	v2



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	x+v2



Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

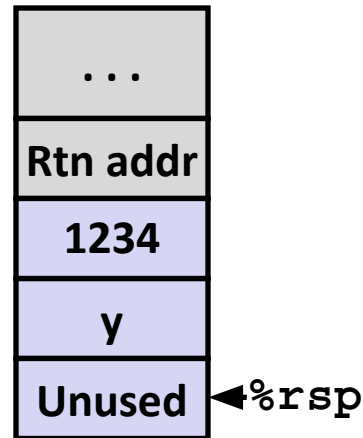
Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

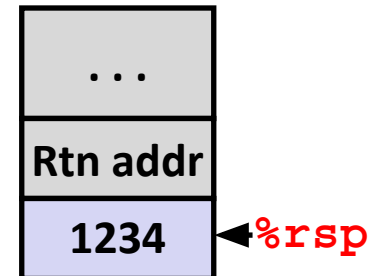
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	x+v2



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	x+v2



Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

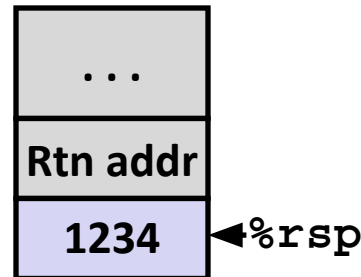
Procedure Assembly Example II

```
long call_incr2(long x) {  
    long v1 = 60637;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     $60637, 8(%rsp)  
    movq     %rdi, %rbx  
    movq     $3000, %rsi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

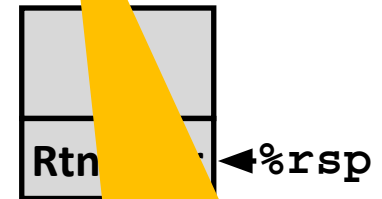
Initial
states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	x
%rax	x+v2



Resulting states

Register	Use(s)
%rdi	&v1
%rsi	y
%rbx	1234
%rax	x+v2



Restore callee-saved
register to original value

Caller-saved: %rdi, %rsi, %rax

Callee-saved: %rbx

Procedure Summary

- **Stack discipline follows call / return pattern**

- “If P calls Q, then Q returns before P” $\Leftrightarrow$ “Stack: Last-In, First-Out”
- Also works for mutual recursion: P calls Q; Q calls P

- **Stack frames allow each function call to have its private storage**

- Saved registers & local variables
- Saved return pointer

- **Register saving conventions prevent one function call from corrupting another’s data**

